

# 计算机组成原理 Project1 实验报告

## 8 位定点卷积-激活-池化链运算器设计

鲁昕宁 2414015

2026 年 4 月 26 日

<https://github.com/SidneyLu/Computer-Organization-and-Design-NKU-CS/tree/master/project1>

## 1 实验目标

本实验面向边缘端 AI 场景，设计并验证一个 8 位定点卷积-激活-池化链运算器。工程目标如下：

- 输入为  $5 \times 5$  的 8 位无符号灰度矩阵，像素范围为  $0 \sim 255$ ；
- 运算流程为“ $5 \times 5$  输入  $\rightarrow 3 \times 3$  卷积层  $\rightarrow$  ReLU 激活层  $\rightarrow 2 \times 2$  平均池化层”；
- 输出为  $2 \times 2$  的 8 位特征值矩阵；
- 在 Vivado 环境下完成模块设计、功能仿真、综合与时序分析，并比较不同优化方案的资源占用与性能。

## 2 实验环境

本工程的开发与验证环境如下：

- FPGA 器件：Xilinx Artix-7 xc7a200tfbg676-2
- RTL 语言：Verilog HDL
- 功能仿真：Vivado 2025.1 / XSIM
- 综合与时序分析：Vivado 批处理脚本 `scripts/run_vivado_multi_synth.tcl`
- 时钟约束：`create_clock -period 5.000 -name clk [get_ports clk]`，即 200 MHz
- I/O 约束：`set_input_delay 0.000` 与 `set_output_delay 0.000`  
波形保留策略也在工程中固化：
- XSIM 原始波形数据库保存在 `build/xsim/waves/*.wdb`；
- 便于二次处理的文本波形保存在 `build/xsim/waves/*.vcd`；
- 报告中的代表性波形图由 `scripts/export_wave_figures.py` 从 VCD 自动导出到 `report/figures/`。

## 3 实验原理

### 3.1 卷积运算

输入图像记为  $X$ ，卷积核记为  $K$ 。本实验采用  $3 \times 3$  卷积核对  $5 \times 5$  输入做 valid 卷积，得到  $3 \times 3$  特征图：

$$Y(i, j) = \sum_{u=0}^2 \sum_{v=0}^2 X(i+u, j+v) \cdot K(u, v), \quad 0 \leq i, j \leq 2$$

其中像素  $X(i, j)$  为 8 位无符号数，卷积核权重  $K(u, v)$  为 8 位有符号数。

### 3.2 ReLU 激活

卷积输出经过 ReLU 激活后，负数被截断为 0，正数保留：

$$R(i, j) = \max(0, Y(i, j))$$

为保证最终结果能安全落回 8 位无符号范围，本工程进一步加入上饱和处理：  
若  $R(i, j) > 255$ ，则输出钳位为 255。

### 3.3 平均池化

对  $3 \times 3$  激活图执行  $2 \times 2$  平均池化，输出  $2 \times 2$  结果：

$$P(i, j) = \left\lfloor \frac{R(i, j) + R(i, j + 1) + R(i + 1, j) + R(i + 1, j + 1)}{4} \right\rfloor, \quad 0 \leq i, j \leq 1$$

由于分母恒为 4，硬件上可直接用右移 2 位实现，因此无需额外的通用除法器。

### 3.4 实验分组

本次实验实现了 5 个独立顶层：

1. 串行: `cnn_compare_base_top`
2. 串行 + BoothWallace 乘法器优化: `cnn_compare_booth_top`
3. BoothWallace 乘法器优化 + SIMD: `cnn_compare_booth_simd_top`
4. BoothWallace 乘法器优化 + SIMD + 流水线: `cnn_compare_booth_simd_pipe_top`
5. 第 4 组基础上换用 DSP: `cnn_compare_booth_simd_pipe_dsp_top`

## 4 里程碑 1：核心模块设计与功能验证

### 4.1 基础模块设计

里程碑 1 首先完成了 4 个基础模块的串行版本：

- `mul8`: 8 位无符号像素与 8 位有符号卷积核的乘法；
- `add8`: 8 位无符号加法；
- `relu_s20`: 20 位有符号卷积结果的 ReLU 与 8 位饱和截断；
- `div4`: 10 位无符号数除以 4 的右移实现。

下面在模块首次介绍的位置给出对应的完整 Verilog 源码。

#### 模块 `mul8` 的 Verilog 源码

对应文件: `../src/baseline/mul8.v`

```
module mul8 (  
    input  wire [7:0] a_u8,  
    input  wire signed [7:0] b_s8,  
    output wire signed [15:0] p_s16
```

```
);
    assign p_s16 = $signed({1'b0, a_u8}) * b_s8;
endmodule
```

### 模块 add8 的 Verilog 源码

对应文件: ../src/baseline/add8.v

```
module add8 (
    input  wire [7:0] a_u8,
    input  wire [7:0] b_u8,
    output wire [7:0] y_u8
);
    assign y_u8 = a_u8 + b_u8;
endmodule
```

### 模块 relu\_s20 的 Verilog 源码

对应文件: ../src/baseline/relu\_s20.v

```
module relu_s20 (
    input  wire signed [19:0] x_s20,
    output wire [7:0] y_u8
);
    wire signed [19:0] x_clip_low;
    assign x_clip_low = (x_s20 < 0) ? 20'sd0 : x_s20;
    assign y_u8 = (x_clip_low > 20'sd255) ? 8'd255 : x_clip_low[7:0];
endmodule
```

### 模块 div4 的 Verilog 源码

对应文件: ../src/baseline/div4.v

```
module div4 (
    input  wire [9:0] x_u10,
    output wire [7:0] y_u8
);
    assign y_u8 = x_u10[9:2];
endmodule
```

在此基础上，又按计划补齐了优化所需模块：

- mul8\_booth\_wallace: 真实的基 4 Booth 编码 + Wallace 树压缩 8 位乘法器；
- simd\_mul8x4: 4 路 BoothWallace lane 封装；
- simd\_add8x4、simd\_relu8x4、simd\_avgpool2x2\_8bit: 分别承担 4 路加法、4 路激活与池化求均值；
- simd\_mul8x4\_dsp 与 mul8\_dsp48e1: 作为第 5 组 DSP 版的乘法实现。

## 模块 mul8\_booth\_wallace 的 Verilog 源码

对应文件: ../src/optimized/mul8\_booth\_wallace.v

```
module mul8_booth_wallace (
    input  wire [7:0] a_u8,
    input  wire signed [7:0] b_s8,
    output wire signed [15:0] p_s16
);

    function automatic signed [16:0] booth_select;
        input [2:0] booth_bits;
        input signed [16:0] multiplicand;
        begin
            case (booth_bits)
                3'b000, 3'b111: booth_select = 17'sd0;
                3'b001, 3'b010: booth_select = multiplicand;
                3'b011: booth_select = multiplicand <<< 1;
                3'b100: booth_select = -(multiplicand <<< 1);
                3'b101, 3'b110: booth_select = -multiplicand;
                default: booth_select = 17'sd0;
            endcase
        end
    endfunction

    wire signed [16:0] a_ext;
    wire [2:0] booth0;
    wire [2:0] booth1;
    wire [2:0] booth2;
    wire [2:0] booth3;

    wire signed [16:0] pp0;
    wire signed [16:0] pp1;
    wire signed [16:0] pp2;
    wire signed [16:0] pp3;

    wire [16:0] csa1_sum;
    wire [16:0] csa1_carry;
    wire [16:0] csa2_sum;
    wire [16:0] csa2_carry;
    wire signed [16:0] prod_ext;

    assign a_ext = $signed({9'd0, a_u8});

    assign booth0 = {b_s8[1], b_s8[0], 1'b0};
    assign booth1 = {b_s8[3], b_s8[2], b_s8[1]};
    assign booth2 = {b_s8[5], b_s8[4], b_s8[3]};
    assign booth3 = {b_s8[7], b_s8[6], b_s8[5]};
```

```

assign pp0 = booth_select(booth0, a_ext);
assign pp1 = booth_select(booth1, a_ext) <<< 2;
assign pp2 = booth_select(booth2, a_ext) <<< 4;
assign pp3 = booth_select(booth3, a_ext) <<< 6;

assign csa1_sum = pp0 ^ pp1 ^ pp2;
assign csa1_carry = ((pp0 & pp1) | (pp0 & pp2) | (pp1 & pp2)) << 1;

assign csa2_sum = csa1_sum ^ csa1_carry ^ pp3;
assign csa2_carry = ((csa1_sum & csa1_carry) | (csa1_sum & pp3) |
↪ (csa1_carry & pp3)) << 1;

assign prod_ext = $signed(csa2_sum) + $signed(csa2_carry);
assign p_s16 = prod_ext[15:0];
endmodule

```

### 模块 simd\_add8x4 的 Verilog 源码

对应文件: ../src/optimized/simd\_add8x4.v

```

module simd_add8x4 (
    input wire [31:0] a_u8x4,
    input wire [31:0] b_u8x4,
    output wire [35:0] y_u9x4
);
    wire [8:0] s0;
    wire [8:0] s1;
    wire [8:0] s2;
    wire [8:0] s3;

    assign s0 = {1'b0, a_u8x4[7:0]} + {1'b0, b_u8x4[7:0]};
    assign s1 = {1'b0, a_u8x4[15:8]} + {1'b0, b_u8x4[15:8]};
    assign s2 = {1'b0, a_u8x4[23:16]} + {1'b0, b_u8x4[23:16]};
    assign s3 = {1'b0, a_u8x4[31:24]} + {1'b0, b_u8x4[31:24]};

    assign y_u9x4 = {s3, s2, s1, s0};
endmodule

```

### 模块 simd\_mul8x4 的 Verilog 源码

对应文件: ../src/optimized/simd\_mul8x4.v

```

module simd_mul8x4 (
    input wire [31:0] a_u8x4,
    input wire [31:0] b_s8x4,
    output wire [63:0] p_s16x4

```

```

);
    wire signed [7:0] b0_s8;
    wire signed [7:0] b1_s8;
    wire signed [7:0] b2_s8;
    wire signed [7:0] b3_s8;

    wire signed [15:0] p0;
    wire signed [15:0] p1;
    wire signed [15:0] p2;
    wire signed [15:0] p3;

    assign b0_s8 = b_s8x4[7:0];
    assign b1_s8 = b_s8x4[15:8];
    assign b2_s8 = b_s8x4[23:16];
    assign b3_s8 = b_s8x4[31:24];

    mul8_booth_wallace u_lane0 (
        .a_u8(a_u8x4[7:0]),
        .b_s8(b0_s8),
        .p_s16(p0)
    );

    mul8_booth_wallace u_lane1 (
        .a_u8(a_u8x4[15:8]),
        .b_s8(b1_s8),
        .p_s16(p1)
    );

    mul8_booth_wallace u_lane2 (
        .a_u8(a_u8x4[23:16]),
        .b_s8(b2_s8),
        .p_s16(p2)
    );

    mul8_booth_wallace u_lane3 (
        .a_u8(a_u8x4[31:24]),
        .b_s8(b3_s8),
        .p_s16(p3)
    );

    assign p_s16x4 = {p3, p2, p1, p0};
endmodule

```

模块 `simd_relu8x4` 的 Verilog 源码

对应文件: `../src/optimized/simd_relu8x4.v`

```

module simd_relu8x4 (
    input  wire signed [79:0] x_s20x4,
    output wire [31:0] y_u8x4
);
    function [7:0] relu_clip_lane;
        input signed [19:0] x_s20;
        begin
            if (x_s20 < 0) begin
                relu_clip_lane = 8'd0;
            end else if (x_s20 > 20'sd255) begin
                relu_clip_lane = 8'd255;
            end else begin
                relu_clip_lane = x_s20[7:0];
            end
        end
    endfunction

    wire [7:0] y0;
    wire [7:0] y1;
    wire [7:0] y2;
    wire [7:0] y3;

    assign y0 = relu_clip_lane(x_s20x4[19:0]);
    assign y1 = relu_clip_lane(x_s20x4[39:20]);
    assign y2 = relu_clip_lane(x_s20x4[59:40]);
    assign y3 = relu_clip_lane(x_s20x4[79:60]);

    assign y_u8x4 = {y3, y2, y1, y0};
endmodule

```

### 模块 simd\_avgpool2x2\_8bit 的 Verilog 源码

对应文件: ../src/optimized/simd\_avgpool2x2\_8bit.v

```

module simd_avgpool2x2_8bit (
    input  wire [31:0] x_u8x4,
    output wire [7:0] y_u8
);
    wire [35:0] pair_sum_u9x4;
    wire [8:0] pair0_u9;
    wire [8:0] pair1_u9;
    wire [9:0] pool_sum_u10;

    simd_add8x4 u_pair_add (
        .a_u8x4({16'd0, x_u8x4[31:24], x_u8x4[15:8]}),
        .b_u8x4({16'd0, x_u8x4[23:16], x_u8x4[7:0]}),
        .y_u9x4(pair_sum_u9x4)
    );

```

```

);

assign pair0_u9 = pair_sum_u9x4[8:0];
assign pair1_u9 = pair_sum_u9x4[17:9];
assign pool_sum_u10 = {1'b0, pair0_u9} + {1'b0, pair1_u9};
assign y_u8 = pool_sum_u10[9:2];
endmodule

```

## 模块 mul8\_dsp48e1 的 Verilog 源码

对应文件: ../src/optimized/mul8\_dsp48e1.v

```

module mul8_dsp48e1 (
    input  wire [7:0] a_u8,
    input  wire signed [7:0] b_s8,
    output wire signed [15:0] p_s16
);
    wire signed [24:0] a_mul_s25;
    wire signed [17:0] b_mul_s18;
    wire signed [47:0] p_dsp_s48;

    // Map the project operand types directly onto the DSP48E1 multiplier:
    // unsigned 8-bit pixel -> positive signed 25-bit operand
    // signed 8-bit weight  -> sign-extended 18-bit operand
    assign a_mul_s25 = {17'd0, a_u8};
    assign b_mul_s18 = {{10{b_s8[7]}}, b_s8};

    DSP48E1 #(
        .ACASCREG(0),
        .ADREG(0),
        .ALUMODEREG(0),
        .AREG(0),
        .A_INPUT("DIRECT"),
        .BCASCREG(0),
        .BREG(0),
        .B_INPUT("DIRECT"),
        .CARRYINREG(0),
        .CARRYINSELREG(0),
        .CREG(0),
        .DREG(0),
        .INMODEREG(0),
        .MREG(0),
        .OPMODEREG(0),
        .PREG(0),
        .USE_DPORT("FALSE"),
        .USE_MULT("MULTIPLY"),
        .USE_SIMD("ONE48")
    )

```

```

) u_dsp48e1 (
    .ACOUT(),
    .BCOUT(),
    .CARRYCASCOUT(),
    .CARRYOUT(),
    .MULTSIGNOUT(),
    .OVERFLOW(),
    .P(p_dsp_s48),
    .PATTERNBDETECT(),
    .PATTERNDETECT(),
    .PCOUT(),
    .UNDERFLOW(),
    .A({5'd0, a_mul_s25}),
    .ACIN(30'd0),
    .ALUMODE(4'b0000),
    .B(b_mul_s18),
    .BCIN(18'd0),
    .C(48'd0),
    .CARRYCASCIN(1'b0),
    .CARRYIN(1'b0),
    .CARRYINSEL(3'b000),
    .CEA1(1'b0),
    .CEA2(1'b0),
    .CEAD(1'b0),
    .CEALUMODE(1'b0),
    .CEB1(1'b0),
    .CEB2(1'b0),
    .CEC(1'b0),
    .CECARRYIN(1'b0),
    .CECTRL(1'b0),
    .CED(1'b0),
    .CEINMODE(1'b0),
    .CEM(1'b0),
    .CEP(1'b0),
    .CLK(1'b0),
    .D(25'd0),
    .INMODE(5'b00000),
    .MULTSIGNIN(1'b0),
    .OPMODE(7'b0000101),
    .PCIN(48'd0),
    .RSTA(1'b0),
    .RSTALLCARRYIN(1'b0),
    .RSTALUMODE(1'b0),
    .RSTB(1'b0),
    .RSTC(1'b0),
    .RSTCTRL(1'b0),

```

```

        .RSTD(1'b0),
        .RSTINMODE(1'b0),
        .RSTM(1'b0),
        .RSTP(1'b0)
    );

    assign p_s16 = p_dsp_s48[15:0];
endmodule

```

### 模块 simd\_mul8x4\_dsp 的 Verilog 源码

对应文件: ../src/optimized/simd\_mul8x4\_dsp.v

```

module simd_mul8x4_dsp (
    input  wire [31:0] a_u8x4,
    input  wire [31:0] b_s8x4,
    output wire [63:0] p_s16x4
);
    wire signed [7:0] b0_s8;
    wire signed [7:0] b1_s8;
    wire signed [7:0] b2_s8;
    wire signed [7:0] b3_s8;

    wire signed [15:0] p0;
    wire signed [15:0] p1;
    wire signed [15:0] p2;
    wire signed [15:0] p3;

    assign b0_s8 = b_s8x4[7:0];
    assign b1_s8 = b_s8x4[15:8];
    assign b2_s8 = b_s8x4[23:16];
    assign b3_s8 = b_s8x4[31:24];

    mul8_dsp48e1 u_lane0 (
        .a_u8(a_u8x4[7:0]),
        .b_s8(b0_s8),
        .p_s16(p0)
    );

    mul8_dsp48e1 u_lane1 (
        .a_u8(a_u8x4[15:8]),
        .b_s8(b1_s8),
        .p_s16(p1)
    );

    mul8_dsp48e1 u_lane2 (
        .a_u8(a_u8x4[23:16]),

```

```

        .b_s8(b2_s8),
        .p_s16(p2)
    );

    mul8_dsp48e1 u_lane3 (
        .a_u8(a_u8x4[31:24]),
        .b_s8(b3_s8),
        .p_s16(p3)
    );

    assign p_s16x4 = {p3, p2, p1, p0};
endmodule

```

## 4.2 模块级仿真

本实验所有仿真均通过 Vivado XSIM 完成。统一命令为：

```

powershell -ExecutionPolicy Bypass -File .\scripts\run_vivado_sim.ps1 -Mode
↪ all

```

其中基础模块验证可单独使用：

```

powershell -ExecutionPolicy Bypass -File .\scripts\run_vivado_sim.ps1 -Mode
↪ basic

```

模块级 testbench 的覆盖范围如表 1 所示。

表 1: 里程碑 1 模块级 testbench

| testbench           | 验证对象             | 覆盖方式                    |
|---------------------|------------------|-------------------------|
| tb_mul8             | mul8             | 穷举 65536 组“像素 × 权重”输入   |
| tb_add8             | add8             | 穷举 65536 组 8 位无符号加法输入   |
| tb_relu_s20         | relu_s20         | 负数、线性区间、饱和区间与边界值        |
| tb_div4             | div4             | 穷举全部 1024 组 10 位输入      |
| tb_mul_compare      | BoothWallace 乘法器 | 与 mul8 做 65536 组穷举一致性对比 |
| tb_simd_mul_compare | SIMD / DSP 乘法器   | lane 等价性、边界值与混合模式对比     |

Vivado 日志显示上述 testbench 均通过：

- tb\_mul8 PASS
- tb\_add8 PASS
- tb\_relu\_s20 PASS
- tb\_div4 PASS
- mul8 vs booth/wallace: PASS
- tb\_simd\_mul\_compare PASS

按照验证对象顺序，下面给出里程碑 1 的全部 testbench 源码。

### testbench tb\_mul8 的 Verilog 源码

对应文件: ../sim/tb\_mul8.v

```
`timescale 1ns/1ps

module tb_mul8;
    reg [7:0] a_u8;
    reg signed [7:0] b_s8;
    wire signed [15:0] p_s16;

    integer ia;
    integer ib;
    reg signed [15:0] expected;

    mul8 dut (
        .a_u8(a_u8),
        .b_s8(b_s8),
        .p_s16(p_s16)
    );

    initial begin
        for (ia = 0; ia < 256; ia = ia + 1) begin
            for (ib = -128; ib < 128; ib = ib + 1) begin
                a_u8 = ia[7:0];
                b_s8 = ib[7:0];
                expected = $signed({1'b0, a_u8}) * b_s8;
                #1;
                if (p_s16 !== expected) begin
                    $display("tb_mul8 FAIL a=%0d b=%0d got=%0d exp=%0d",
                        ia, ib, p_s16, expected);
                    $finish;
                end
            end
        end

        $display("tb_mul8 PASS");
        $finish;
    end
endmodule
```

### testbench tb\_add8 的 Verilog 源码

对应文件: ../sim/tb\_add8.v

```

`timescale 1ns/1ps

module tb_add8;
    reg [7:0] a_u8;
    reg [7:0] b_u8;
    wire [7:0] y_u8;

    integer ia;
    integer ib;
    reg [8:0] expected;

    add8 dut (
        .a_u8(a_u8),
        .b_u8(b_u8),
        .y_u8(y_u8)
    );

    initial begin
        for (ia = 0; ia < 256; ia = ia + 1) begin
            for (ib = 0; ib < 256; ib = ib + 1) begin
                a_u8 = ia[7:0];
                b_u8 = ib[7:0];
                expected = a_u8 + b_u8;
                #1;
                if (y_u8 != expected[7:0]) begin
                    $display("tb_add8 FAIL a=%0d b=%0d got=%0d exp=%0d",
                        ia, ib, y_u8, expected[7:0]);
                    $finish;
                end
            end
        end

        $display("tb_add8 PASS");
        $finish;
    end
endmodule

```

## testbench tb\_relu\_s20 的 Verilog 源码

对应文件: ../sim/tb\_relu\_s20.v

```

`timescale 1ns/1ps

module tb_relu_s20;
    reg signed [19:0] x_s20;
    wire [7:0] y_u8;

```

```

integer iv;
integer expected;

relu_s20 dut (
    .x_s20(x_s20),
    .y_u8(y_u8)
);

task automatic check_value;
    input integer val;
    input integer exp;
    begin
        x_s20 = val;
        #1;
        if (y_u8 !== exp[7:0]) begin
            $display("tb_relu_s20 FAIL x=%0d got=%0d exp=%0d", val,
                ↪ y_u8, exp);
            $finish;
        end
    end
endtask

initial begin
    check_value(-524288, 0);
    check_value(-300, 0);
    check_value(-1, 0);
    check_value(0, 0);
    check_value(1, 1);
    check_value(127, 127);
    check_value(255, 255);
    check_value(256, 255);
    check_value(1024, 255);
    check_value(524287, 255);

    for (iv = -300; iv <= 600; iv = iv + 1) begin
        if (iv < 0) begin
            expected = 0;
        end else if (iv > 255) begin
            expected = 255;
        end else begin
            expected = iv;
        end
        check_value(iv, expected);
    end

    $display("tb_relu_s20 PASS");
end

```

```

        $finish;
    end
endmodule

```

### testbench tb\_div4 的 Verilog 源码

对应文件: ../sim/tb\_div4.v

```

`timescale 1ns/1ps

module tb_div4;
    reg [9:0] x_u10;
    wire [7:0] y_u8;

    integer ix;
    reg [7:0] expected;

    div4 dut (
        .x_u10(x_u10),
        .y_u8(y_u8)
    );

    initial begin
        for (ix = 0; ix < 1024; ix = ix + 1) begin
            x_u10 = ix[9:0];
            expected = ix[9:2];
            #1;
            if (y_u8 != expected) begin
                $display("tb_div4 FAIL x=%0d got=%0d exp=%0d", ix, y_u8,
                    ↪ expected);
                $finish;
            end
        end
        $display("tb_div4 PASS");
        $finish;
    end
endmodule

```

### testbench tb\_mul\_compare 的 Verilog 源码

对应文件: ../sim/tb\_mul\_compare.v

```

`timescale 1ns/1ps

module tb_mul_compare;
    reg [7:0] a_u8;

```

```

    reg signed [7:0] b_s8;
    wire signed [15:0] p_base;
    wire signed [15:0] p_booth;
    integer ia;
    integer ib;

    mul8 u_base (
        .a_u8(a_u8),
        .b_s8(b_s8),
        .p_s16(p_base)
    );

    mul8_booth_wallace u_booth (
        .a_u8(a_u8),
        .b_s8(b_s8),
        .p_s16(p_booth)
    );

    initial begin
        for (ia = 0; ia < 256; ia = ia + 1) begin
            for (ib = -128; ib < 128; ib = ib + 1) begin
                a_u8 = ia[7:0];
                b_s8 = ib[7:0];
                #1;
                if (p_base != p_booth) begin
                    $display("MISMATCH a=%0d b=%0d base=%0d booth=%0d", ia,
                        ↪ ib, p_base, p_booth);
                    $finish;
                end
            end
        end

        $display("mul8 vs booth/wallace: PASS");
        $finish;
    end
endmodule

```

### testbench tb\_simd\_mul\_compare 的 Verilog 源码

对应文件: ../sim/tb\_simd\_mul\_compare.v

```
`timescale 1ns/1ps
```

```

module tb_simd_mul_compare;
    reg [31:0] a_u8x4;
    reg [31:0] b_s8x4;
    wire [63:0] p_booth_s16x4;
    wire [63:0] p_dsp_s16x4;

```

```

integer ia;
integer ib;
integer lane;
integer expected;

simd_mul8x4 u_booth (
    .a_u8x4(a_u8x4),
    .b_s8x4(b_s8x4),
    .p_s16x4(p_booth_s16x4)
);

simd_mul8x4_dsp u_dsp (
    .a_u8x4(a_u8x4),
    .b_s8x4(b_s8x4),
    .p_s16x4(p_dsp_s16x4)
);

task automatic check_lane_equal;
    input integer lane_idx;
    input integer exp_val;
    reg signed [15:0] booth_lane;
    reg signed [15:0] dsp_lane;
    begin
        booth_lane = p_booth_s16x4[lane_idx*16 +: 16];
        dsp_lane = p_dsp_s16x4[lane_idx*16 +: 16];
        if (booth_lane !== exp_val || dsp_lane !== exp_val || booth_lane
            ↪ !== dsp_lane) begin
            $display("tb_simd_mul_compare FAIL lane=%0d got_booth=%0d
            ↪ got_dsp=%0d exp=%0d",
                lane_idx, booth_lane, dsp_lane, exp_val);
            $finish;
        end
    end
endtask

task automatic check_vector_pattern;
    input [31:0] a_vec;
    input [31:0] b_vec;
    reg signed [15:0] exp0;
    reg signed [15:0] exp1;
    reg signed [15:0] exp2;
    reg signed [15:0] exp3;
    begin
        a_u8x4 = a_vec;
        b_s8x4 = b_vec;

```

```

        exp0 = $signed({1'b0, a_vec[7:0]}) * $signed(b_vec[7:0]);
        exp1 = $signed({1'b0, a_vec[15:8]}) * $signed(b_vec[15:8]);
        exp2 = $signed({1'b0, a_vec[23:16]}) * $signed(b_vec[23:16]);
        exp3 = $signed({1'b0, a_vec[31:24]}) * $signed(b_vec[31:24]);
        #1;
        check_lane_equal(0, exp0);
        check_lane_equal(1, exp1);
        check_lane_equal(2, exp2);
        check_lane_equal(3, exp3);
    end
endtask

initial begin
    for (ia = 0; ia < 256; ia = ia + 1) begin
        for (ib = -128; ib < 128; ib = ib + 1) begin
            a_u8x4 = {ia[7:0], ia[7:0], ia[7:0], ia[7:0]};
            b_s8x4 = {ib[7:0], ib[7:0], ib[7:0], ib[7:0]};
            expected = ia * ib;
            #1;
            for (lane = 0; lane < 4; lane = lane + 1) begin
                check_lane_equal(lane, expected);
            end
        end
    end

    check_vector_pattern({8'd255, 8'd128, 8'd17, 8'd3},
                        {8'sd127, -8'sd64, 8'sd5, -8'sd7});
    check_vector_pattern({8'd0, 8'd1, 8'd200, 8'd255},
                        {-8'sd128, 8'sd0, -8'sd1, 8'sd1});
    check_vector_pattern({8'd12, 8'd34, 8'd56, 8'd78},
                        {8'sd9, -8'sd10, 8'sd11, -8'sd12});

    $display("tb_simd_mul_compare PASS");
    $finish;
end
endmodule

```

### 4.3 代表性基础模块波形

下面给出 4 个基础模块的代表性波形，按模块验证顺序依次展示。

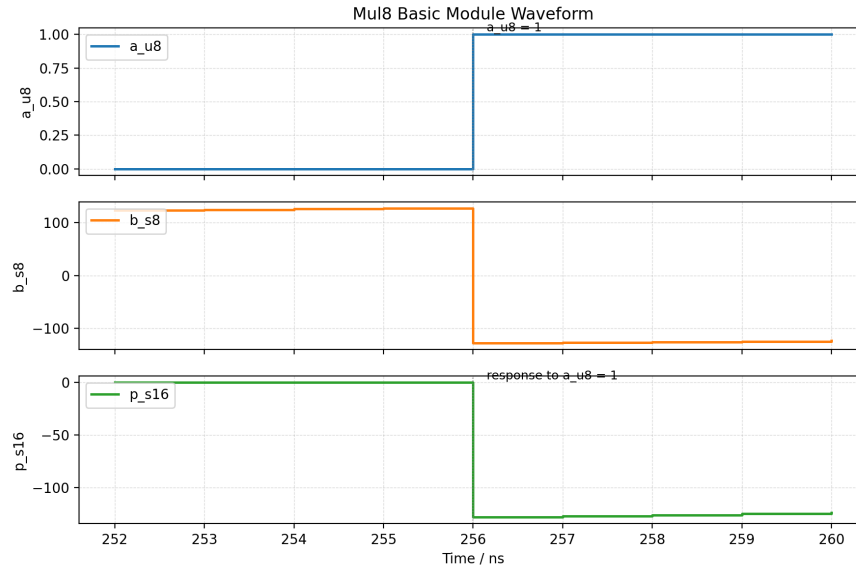


图 1: 基础模块波形: mul8 的乘法响应

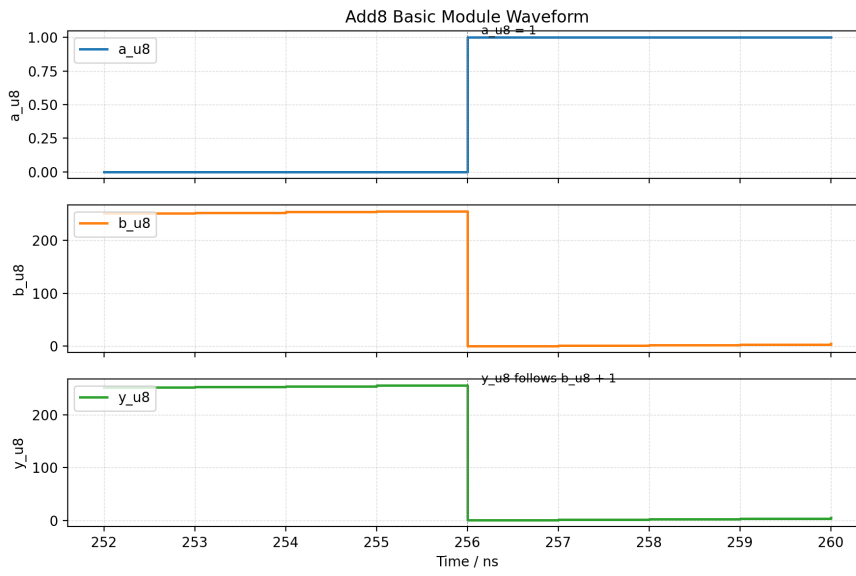


图 2: 基础模块波形: add8 的加法响应

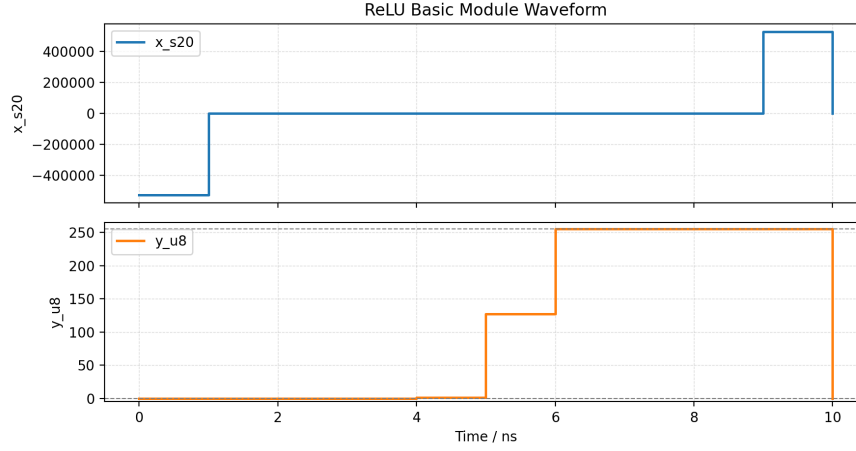


图 3: 基础模块波形: `relu_s20` 的截断与饱和行为

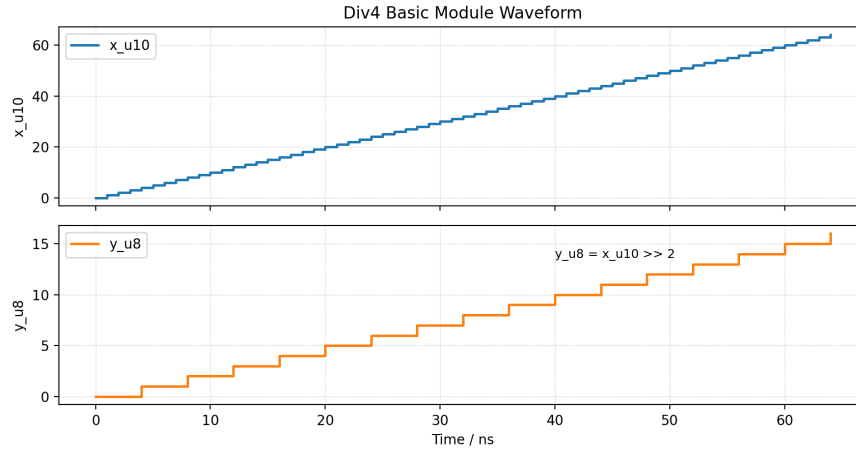


图 4: 基础模块波形: `div4` 的右移除 4 行为

## 5 里程碑 2: 链式运算器集成与系统仿真

### 5.1 链式运算器的独立实现

本次 5 个实验组全部以独立顶层模块存在, 结构差异如下:

- 第 1 组与第 2 组共享串行状态机框架, 区别仅在于卷积阶段使用 `mul8` 还是 `mul8_booth_wallace`;
- 第 3 组采用独立的 BoothWallace + SIMD 数据通路, 卷积结果、ReLU 与池化全部在同一条 32 位 SIMD 路径中完成;
- 第 4 组在第 3 组基础上加入固定 3 级数据流水, 级间顺序为“卷积结果寄存 → ReLU 结果寄存 → 池化/输出寄存”;
- 第 5 组复用第 4 组流水结构, 仅将第 1 级 SIMD 乘法阵列替换为显式

DSP48E1 实现。

### 顶层 cnn\_compare\_base\_top 的 Verilog 源码

对应文件: ../src/baseline/cnn\_compare\_base\_top.v

```
module cnn_compare_base_top (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output wire done,
    output wire [31:0] out_flat_u8x4
);
    cnn_chain_core_base u_base (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .use_booth(1'b0),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .done(done),
        .out_flat_u8x4(out_flat_u8x4)
    );
endmodule
```

### 顶层 cnn\_compare\_booth\_top 的 Verilog 源码

对应文件: ../src/optimized/cnn\_compare\_booth\_top.v

```
module cnn_compare_booth_top (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output wire done,
    output wire [31:0] out_flat_u8x4
);
    cnn_chain_core_base u_booth_base (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .use_booth(1'b1),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),

```

```

        .done(done),
        .out_flat_u8x4(out_flat_u8x4)
    );
endmodule

```

## 模块 cnn\_chain\_core\_base 的 Verilog 源码

对应文件: ../src/baseline/cnn\_chain\_core\_base.v

```

module cnn_chain_core_base (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire use_booth,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output reg  done,
    output reg  [31:0] out_flat_u8x4
);
    localparam S_IDLE = 3'd0;
    localparam S_CONV = 3'd1;
    localparam S_RELU = 3'd2;
    localparam S_POOL = 3'd3;
    localparam S_DONE = 3'd4;

    reg [2:0] state;
    reg [1:0] ox;
    reg [1:0] oy;
    reg [3:0] k;
    reg [3:0] relu_idx;
    reg [1:0] pool_idx;

    reg signed [19:0] conv_map [0:8];
    reg [7:0] relu_map [0:8];

    wire [7:0] pix_cur;
    wire signed [7:0] ker_cur;
    wire signed [15:0] prod_base;
    wire signed [15:0] prod_booth;
    wire signed [15:0] prod_cur;
    wire [7:0] relu_cur;

    wire [7:0] p0;
    wire [7:0] p1;
    wire [7:0] p2;
    wire [7:0] p3;
    wire [9:0] pool_sum;

```

```

wire [7:0] pool_avg;

integer loop_i;
wire [3:0] conv_idx;
wire [5:0] pix_idx;
wire [3:0] base_idx;
wire [1:0] pool_r;
wire [1:0] pool_c;

wire [1:0] ix;
wire [1:0] iy;
assign ix = k % 3;
assign iy = k / 3;
assign conv_idx = oy * 3 + ox;
assign pix_idx = (oy + iy) * 5 + (ox + ix);
assign p0 = relu_map[base_idx];
assign p1 = relu_map[base_idx + 1];
assign p2 = relu_map[base_idx + 3];
assign p3 = relu_map[base_idx + 4];
assign pool_sum = {2'b00, p0} + {2'b00, p1} + {2'b00, p2} + {2'b00, p3};

assign pix_cur = img_flat_u8x25[pix_idx*8 +: 8];
assign ker_cur = ker_flat_s8x9[k*8 +: 8];

mul8 u_mul8 (
    .a_u8(pix_cur),
    .b_s8(ker_cur),
    .p_s16(prod_base)
);

mul8_booth_wallace u_mul8_booth (
    .a_u8(pix_cur),
    .b_s8(ker_cur),
    .p_s16(prod_booth)
);

assign prod_cur = use_booth ? prod_booth : prod_base;

relu_s20 u_relu (
    .x_s20(conv_map[relu_idx]),
    .y_u8(relu_cur)
);

div4 u_div4 (
    .x_u10(pool_sum),
    .y_u8(pool_avg)

```

```

);

assign pool_r = pool_idx[1];
assign pool_c = pool_idx[0];
assign base_idx = pool_r * 3 + pool_c;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= S_IDLE;
        ox <= 2'd0;
        oy <= 2'd0;
        k <= 4'd0;
        relu_idx <= 4'd0;
        pool_idx <= 2'd0;
        done <= 1'b0;
        out_flat_u8x4 <= 32'd0;
        for (loop_i = 0; loop_i < 9; loop_i = loop_i + 1) begin
            conv_map[loop_i] <= 20'sd0;
            relu_map[loop_i] <= 8'd0;
        end
    end else begin
        done <= 1'b0;
        case (state)
            S_IDLE: begin
                if (start) begin
                    ox <= 2'd0;
                    oy <= 2'd0;
                    k <= 4'd0;
                    relu_idx <= 4'd0;
                    pool_idx <= 2'd0;
                    out_flat_u8x4 <= 32'd0;
                    for (loop_i = 0; loop_i < 9; loop_i = loop_i + 1)
                        ↪ begin
                            conv_map[loop_i] <= 20'sd0;
                            relu_map[loop_i] <= 8'd0;
                        end
                    state <= S_CONV;
                end
            end
            S_CONV: begin
                conv_map[conv_idx] <= conv_map[conv_idx] +
                    ↪ $signed(prod_cur);
                if (k == 4'd8) begin
                    k <= 4'd0;
                    if (ox == 2'd2) begin
                        ox <= 2'd0;

```

```

        if (oy == 2'd2) begin
            oy <= 2'd0;
            state <= S_RELU;
        end else begin
            oy <= oy + 2'd1;
        end
    end else begin
        ox <= ox + 2'd1;
    end
end else begin
    k <= k + 4'd1;
end
end
S_RELU: begin
    relu_map[relu_idx] <= relu_cur;
    if (relu_idx == 4'd8) begin
        relu_idx <= 4'd0;
        state <= S_POOL;
    end else begin
        relu_idx <= relu_idx + 4'd1;
    end
end
end
S_POOL: begin
    out_flat_u8x4[pool_idx*8 +: 8] <= pool_avg;
    if (pool_idx == 2'd3) begin
        pool_idx <= 2'd0;
        state <= S_DONE;
    end else begin
        pool_idx <= pool_idx + 2'd1;
    end
end
end
S_DONE: begin
    done <= 1'b1;
    state <= S_IDLE;
end
default: begin
    state <= S_IDLE;
end
endcase
end
end
endmodule

```

## 顶层 cnn\_compare\_booth\_simd\_top 的 Verilog 源码

对应文件: ../src/optimized/cnn\_compare\_booth\_simd\_top.v

```
module cnn_compare_booth_simd_top (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output wire done,
    output wire [31:0] out_flat_u8x4
);
    cnn_chain_core_booth_simd u_booth_simd (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .done(done),
        .out_flat_u8x4(out_flat_u8x4)
    );
endmodule
```

## 顶层 cnn\_compare\_booth\_simd\_pipe\_top 的 Verilog 源码

对应文件: ../src/optimized/cnn\_compare\_booth\_simd\_pipe\_top.v

```
module cnn_compare_booth_simd_pipe_top (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output wire done,
    output wire [31:0] out_flat_u8x4
);
    cnn_chain_core_booth_simd_pipe u_booth_simd_pipe (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .done(done),
        .out_flat_u8x4(out_flat_u8x4)
    );
endmodule
```

## 顶层 cnn\_compare\_booth\_simd\_pipe\_dsp\_top 的 Verilog 源码

对应文件: ../src/optimized/cnn\_compare\_booth\_simd\_pipe\_dsp\_top.v

```
module cnn_compare_booth_simd_pipe_dsp_top (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output wire done,
    output wire [31:0] out_flat_u8x4
);
    cnn_chain_core_booth_simd_pipe_dsp u_booth_simd_pipe_dsp (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .done(done),
        .out_flat_u8x4(out_flat_u8x4)
    );
endmodule
```

## 模块 cnn\_chain\_core\_booth\_simd 的 Verilog 源码

对应文件: ../src/optimized/cnn\_chain\_core\_booth\_simd.v

```
module cnn_chain_core_booth_simd (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output reg  done,
    output reg  [31:0] out_flat_u8x4
);
    wire signed [79:0] conv_blk0_s20x4;
    wire signed [79:0] conv_blk1_s20x4;
    wire signed [79:0] conv_blk2_s20x4;

    wire [31:0] relu_blk0_u8x4;
    wire [31:0] relu_blk1_u8x4;
    wire [31:0] relu_blk2_u8x4;

    wire [7:0] relu0;
    wire [7:0] relu1;
```

```

wire [7:0] relu2;
wire [7:0] relu3;
wire [7:0] relu4;
wire [7:0] relu5;
wire [7:0] relu6;
wire [7:0] relu7;
wire [7:0] relu8;

wire [7:0] pool0_u8;
wire [7:0] pool1_u8;
wire [7:0] pool2_u8;
wire [7:0] pool3_u8;
wire [31:0] out_comb_u8x4;

cnn_conv3x3_simd_core #(
    .USE_DSP(1'b0)
) u_conv (
    .img_flat_u8x25(img_flat_u8x25),
    .ker_flat_s8x9(ker_flat_s8x9),
    .conv_blk0_s20x4(conv_blk0_s20x4),
    .conv_blk1_s20x4(conv_blk1_s20x4),
    .conv_blk2_s20x4(conv_blk2_s20x4)
);

simd_relu8x4 u_relu_blk0 (
    .x_s20x4(conv_blk0_s20x4),
    .y_u8x4(relu_blk0_u8x4)
);

simd_relu8x4 u_relu_blk1 (
    .x_s20x4(conv_blk1_s20x4),
    .y_u8x4(relu_blk1_u8x4)
);

simd_relu8x4 u_relu_blk2 (
    .x_s20x4(conv_blk2_s20x4),
    .y_u8x4(relu_blk2_u8x4)
);

assign relu0 = relu_blk0_u8x4[7:0];
assign relu1 = relu_blk0_u8x4[15:8];
assign relu2 = relu_blk0_u8x4[23:16];
assign relu3 = relu_blk0_u8x4[31:24];
assign relu4 = relu_blk1_u8x4[7:0];
assign relu5 = relu_blk1_u8x4[15:8];
assign relu6 = relu_blk1_u8x4[23:16];

```

```

assign relu7 = relu_blk1_u8x4[31:24];
assign relu8 = relu_blk2_u8x4[7:0];

simd_avgpool2x2_8bit u_pool0 (
    .x_u8x4({relu4, relu3, relu1, relu0}),
    .y_u8(pool0_u8)
);

simd_avgpool2x2_8bit u_pool1 (
    .x_u8x4({relu5, relu4, relu2, relu1}),
    .y_u8(pool1_u8)
);

simd_avgpool2x2_8bit u_pool2 (
    .x_u8x4({relu7, relu6, relu4, relu3}),
    .y_u8(pool2_u8)
);

simd_avgpool2x2_8bit u_pool3 (
    .x_u8x4({relu8, relu7, relu5, relu4}),
    .y_u8(pool3_u8)
);

assign out_comb_u8x4 = {pool3_u8, pool2_u8, pool1_u8, pool0_u8};

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        done <= 1'b0;
        out_flat_u8x4 <= 32'd0;
    end else begin
        done <= start;
        if (start) begin
            out_flat_u8x4 <= out_comb_u8x4;
        end
    end
end

endmodule

```

模块 `cnn_chain_core_booth_simd_pipe` 的 Verilog 源码

对应文件: `../src/optimized/cnn_chain_core_booth_simd_pipe.v`

```

module cnn_chain_core_booth_simd_pipe (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,

```

```

    input  wire signed [71:0] ker_flat_s8x9,
    output reg  done,
    output reg  [31:0] out_flat_u8x4
);
    reg [2:0] vld_pipe;

    reg signed [79:0] conv_blk0_reg;
    reg signed [79:0] conv_blk1_reg;
    reg signed [79:0] conv_blk2_reg;

    reg [31:0] relu_blk0_reg;
    reg [31:0] relu_blk1_reg;
    reg [31:0] relu_blk2_reg;

    wire signed [79:0] conv_blk0_s20x4;
    wire signed [79:0] conv_blk1_s20x4;
    wire signed [79:0] conv_blk2_s20x4;

    wire [31:0] relu_blk0_stage2;
    wire [31:0] relu_blk1_stage2;
    wire [31:0] relu_blk2_stage2;

    wire [7:0] relu0;
    wire [7:0] relu1;
    wire [7:0] relu2;
    wire [7:0] relu3;
    wire [7:0] relu4;
    wire [7:0] relu5;
    wire [7:0] relu6;
    wire [7:0] relu7;
    wire [7:0] relu8;

    wire [7:0] pool0_u8;
    wire [7:0] pool1_u8;
    wire [7:0] pool2_u8;
    wire [7:0] pool3_u8;
    wire [31:0] out_stage3_u8x4;

    cnn_conv3x3_simd_core #(
        .USE_DSP(1'b0)
    ) u_conv (
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .conv_blk0_s20x4(conv_blk0_s20x4),
        .conv_blk1_s20x4(conv_blk1_s20x4),
        .conv_blk2_s20x4(conv_blk2_s20x4)

```

```

);

simd_relu8x4 u_relu_blk0 (
    .x_s20x4(conv_blk0_reg),
    .y_u8x4(relu_blk0_stage2)
);

simd_relu8x4 u_relu_blk1 (
    .x_s20x4(conv_blk1_reg),
    .y_u8x4(relu_blk1_stage2)
);

simd_relu8x4 u_relu_blk2 (
    .x_s20x4(conv_blk2_reg),
    .y_u8x4(relu_blk2_stage2)
);

assign relu0 = relu_blk0_reg[7:0];
assign relu1 = relu_blk0_reg[15:8];
assign relu2 = relu_blk0_reg[23:16];
assign relu3 = relu_blk0_reg[31:24];
assign relu4 = relu_blk1_reg[7:0];
assign relu5 = relu_blk1_reg[15:8];
assign relu6 = relu_blk1_reg[23:16];
assign relu7 = relu_blk1_reg[31:24];
assign relu8 = relu_blk2_reg[7:0];

simd_avgpool2x2_8bit u_pool0 (
    .x_u8x4({relu4, relu3, relu1, relu0}),
    .y_u8(pool0_u8)
);

simd_avgpool2x2_8bit u_pool1 (
    .x_u8x4({relu5, relu4, relu2, relu1}),
    .y_u8(pool1_u8)
);

simd_avgpool2x2_8bit u_pool2 (
    .x_u8x4({relu7, relu6, relu4, relu3}),
    .y_u8(pool2_u8)
);

simd_avgpool2x2_8bit u_pool3 (
    .x_u8x4({relu8, relu7, relu5, relu4}),
    .y_u8(pool3_u8)
);

```

```

assign out_stage3_u8x4 = {pool3_u8, pool2_u8, pool1_u8, pool0_u8};

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        vld_pipe <= 3'b000;
        conv_blk0_reg <= 80'sd0;
        conv_blk1_reg <= 80'sd0;
        conv_blk2_reg <= 80'sd0;
        relu_blk0_reg <= 32'd0;
        relu_blk1_reg <= 32'd0;
        relu_blk2_reg <= 32'd0;
        out_flat_u8x4 <= 32'd0;
        done <= 1'b0;
    end else begin
        vld_pipe <= {vld_pipe[1:0], start};
        done <= vld_pipe[1];

        if (start) begin
            conv_blk0_reg <= conv_blk0_s20x4;
            conv_blk1_reg <= conv_blk1_s20x4;
            conv_blk2_reg <= conv_blk2_s20x4;
        end

        if (vld_pipe[0]) begin
            relu_blk0_reg <= relu_blk0_stage2;
            relu_blk1_reg <= relu_blk1_stage2;
            relu_blk2_reg <= relu_blk2_stage2;
        end

        if (vld_pipe[1]) begin
            out_flat_u8x4 <= out_stage3_u8x4;
        end
    end
end
endmodule

```

模块 `cnn_chain_core_booth_simd_pipe_dsp` 的 Verilog 源码

对应文件: `../src/optimized/cnn_chain_core_booth_simd_pipe_dsp.v`

```

module cnn_chain_core_booth_simd_pipe_dsp (
    input  wire clk,
    input  wire rst_n,
    input  wire start,
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,

```

```

    output reg  done,
    output reg  [31:0] out_flat_u8x4
);

    reg [2:0] vld_pipe;

    reg signed [79:0] conv_blk0_reg;
    reg signed [79:0] conv_blk1_reg;
    reg signed [79:0] conv_blk2_reg;

    reg [31:0] relu_blk0_reg;
    reg [31:0] relu_blk1_reg;
    reg [31:0] relu_blk2_reg;

    wire signed [79:0] conv_blk0_s20x4;
    wire signed [79:0] conv_blk1_s20x4;
    wire signed [79:0] conv_blk2_s20x4;

    wire [31:0] relu_blk0_stage2;
    wire [31:0] relu_blk1_stage2;
    wire [31:0] relu_blk2_stage2;

    wire [7:0] relu0;
    wire [7:0] relu1;
    wire [7:0] relu2;
    wire [7:0] relu3;
    wire [7:0] relu4;
    wire [7:0] relu5;
    wire [7:0] relu6;
    wire [7:0] relu7;
    wire [7:0] relu8;

    wire [7:0] pool0_u8;
    wire [7:0] pool1_u8;
    wire [7:0] pool2_u8;
    wire [7:0] pool3_u8;
    wire [31:0] out_stage3_u8x4;

    cnn_conv3x3_simd_core #(
        .USE_DSP(1'b1)
    ) u_conv (
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .conv_blk0_s20x4(conv_blk0_s20x4),
        .conv_blk1_s20x4(conv_blk1_s20x4),
        .conv_blk2_s20x4(conv_blk2_s20x4)
    );

```

```

simd_relu8x4 u_relu_blk0 (
    .x_s20x4(conv_blk0_reg),
    .y_u8x4(relu_blk0_stage2)
);

simd_relu8x4 u_relu_blk1 (
    .x_s20x4(conv_blk1_reg),
    .y_u8x4(relu_blk1_stage2)
);

simd_relu8x4 u_relu_blk2 (
    .x_s20x4(conv_blk2_reg),
    .y_u8x4(relu_blk2_stage2)
);

assign relu0 = relu_blk0_reg[7:0];
assign relu1 = relu_blk0_reg[15:8];
assign relu2 = relu_blk0_reg[23:16];
assign relu3 = relu_blk0_reg[31:24];
assign relu4 = relu_blk1_reg[7:0];
assign relu5 = relu_blk1_reg[15:8];
assign relu6 = relu_blk1_reg[23:16];
assign relu7 = relu_blk1_reg[31:24];
assign relu8 = relu_blk2_reg[7:0];

simd_avgpool2x2_8bit u_pool0 (
    .x_u8x4({relu4, relu3, relu1, relu0}),
    .y_u8(pool0_u8)
);

simd_avgpool2x2_8bit u_pool1 (
    .x_u8x4({relu5, relu4, relu2, relu1}),
    .y_u8(pool1_u8)
);

simd_avgpool2x2_8bit u_pool2 (
    .x_u8x4({relu7, relu6, relu4, relu3}),
    .y_u8(pool2_u8)
);

simd_avgpool2x2_8bit u_pool3 (
    .x_u8x4({relu8, relu7, relu5, relu4}),
    .y_u8(pool3_u8)
);

```

```

assign out_stage3_u8x4 = {pool3_u8, pool2_u8, pool1_u8, pool0_u8};

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        vld_pipe <= 3'b000;
        conv_blk0_reg <= 80'sd0;
        conv_blk1_reg <= 80'sd0;
        conv_blk2_reg <= 80'sd0;
        relu_blk0_reg <= 32'd0;
        relu_blk1_reg <= 32'd0;
        relu_blk2_reg <= 32'd0;
        out_flat_u8x4 <= 32'd0;
        done <= 1'b0;
    end else begin
        vld_pipe <= {vld_pipe[1:0], start};
        done <= vld_pipe[1];

        if (start) begin
            conv_blk0_reg <= conv_blk0_s20x4;
            conv_blk1_reg <= conv_blk1_s20x4;
            conv_blk2_reg <= conv_blk2_s20x4;
        end

        if (vld_pipe[0]) begin
            relu_blk0_reg <= relu_blk0_stage2;
            relu_blk1_reg <= relu_blk1_stage2;
            relu_blk2_reg <= relu_blk2_stage2;
        end

        if (vld_pipe[1]) begin
            out_flat_u8x4 <= out_stage3_u8x4;
        end
    end
end
endmodule

```

### 共享卷积核心 cnn\_conv3x3\_simd\_core 的 Verilog 源码

对应文件: ../src/optimized/cnn\_conv3x3\_simd\_core.v

```

module cnn_conv3x3_simd_core #(
    parameter USE_DSP = 1'b0
) (
    input  wire [199:0] img_flat_u8x25,
    input  wire signed [71:0] ker_flat_s8x9,
    output wire signed [79:0] conv_blk0_s20x4,
    output wire signed [79:0] conv_blk1_s20x4,

```

```

output wire signed [79:0] conv_blk2_s20x4
);
wire [31:0] pix_blk0 [0:8];
wire [31:0] pix_blk1 [0:8];
wire [31:0] pix_blk2 [0:8];
wire [31:0] ker_pack [0:8];

wire [63:0] prod_blk0 [0:8];
wire [63:0] prod_blk1 [0:8];
wire [63:0] prod_blk2 [0:8];

wire signed [19:0] conv0;
wire signed [19:0] conv1;
wire signed [19:0] conv2;
wire signed [19:0] conv3;
wire signed [19:0] conv4;
wire signed [19:0] conv5;
wire signed [19:0] conv6;
wire signed [19:0] conv7;
wire signed [19:0] conv8;

function signed [19:0] lane20;
    input [63:0] lane_bus;
    input integer lane_idx;
    reg signed [15:0] lane16;
    begin
        case (lane_idx)
            0: lane16 = lane_bus[15:0];
            1: lane16 = lane_bus[31:16];
            2: lane16 = lane_bus[47:32];
            default: lane16 = lane_bus[63:48];
        endcase
        lane20 = {{4{lane16[15]}}, lane16};
    end
endfunction

assign ker_pack[0] = {4{ker_flat_s8x9[7:0]}};
assign ker_pack[1] = {4{ker_flat_s8x9[15:8]}};
assign ker_pack[2] = {4{ker_flat_s8x9[23:16]}};
assign ker_pack[3] = {4{ker_flat_s8x9[31:24]}};
assign ker_pack[4] = {4{ker_flat_s8x9[39:32]}};
assign ker_pack[5] = {4{ker_flat_s8x9[47:40]}};
assign ker_pack[6] = {4{ker_flat_s8x9[55:48]}};
assign ker_pack[7] = {4{ker_flat_s8x9[63:56]}};
assign ker_pack[8] = {4{ker_flat_s8x9[71:64]}};

```

```

assign pix_blk0[0] = {img_flat_u8x25[47:40], img_flat_u8x25[23:16],
  ↪ img_flat_u8x25[15:8], img_flat_u8x25[7:0]};
assign pix_blk0[1] = {img_flat_u8x25[55:48], img_flat_u8x25[31:24],
  ↪ img_flat_u8x25[23:16], img_flat_u8x25[15:8]};
assign pix_blk0[2] = {img_flat_u8x25[63:56], img_flat_u8x25[39:32],
  ↪ img_flat_u8x25[31:24], img_flat_u8x25[23:16]};
assign pix_blk0[3] = {img_flat_u8x25[87:80], img_flat_u8x25[63:56],
  ↪ img_flat_u8x25[55:48], img_flat_u8x25[47:40]};
assign pix_blk0[4] = {img_flat_u8x25[95:88], img_flat_u8x25[71:64],
  ↪ img_flat_u8x25[63:56], img_flat_u8x25[55:48]};
assign pix_blk0[5] = {img_flat_u8x25[103:96], img_flat_u8x25[79:72],
  ↪ img_flat_u8x25[71:64], img_flat_u8x25[63:56]};
assign pix_blk0[6] = {img_flat_u8x25[127:120],img_flat_u8x25[103:96],
  ↪ img_flat_u8x25[95:88], img_flat_u8x25[87:80]};
assign pix_blk0[7] = {img_flat_u8x25[135:128],img_flat_u8x25[111:104],i
  ↪ mg_flat_u8x25[103:96],
  ↪ img_flat_u8x25[95:88]};
assign pix_blk0[8] = {img_flat_u8x25[143:136],img_flat_u8x25[119:112],i
  ↪ mg_flat_u8x25[111:104],img_flat_u8x25[103:96]};

assign pix_blk1[0] = {img_flat_u8x25[95:88], img_flat_u8x25[87:80],
  ↪ img_flat_u8x25[63:56], img_flat_u8x25[55:48]};
assign pix_blk1[1] = {img_flat_u8x25[103:96], img_flat_u8x25[95:88],
  ↪ img_flat_u8x25[71:64], img_flat_u8x25[63:56]};
assign pix_blk1[2] = {img_flat_u8x25[111:104],img_flat_u8x25[103:96],
  ↪ img_flat_u8x25[79:72], img_flat_u8x25[71:64]};
assign pix_blk1[3] = {img_flat_u8x25[135:128],img_flat_u8x25[127:120],i
  ↪ mg_flat_u8x25[103:96],
  ↪ img_flat_u8x25[95:88]};
assign pix_blk1[4] = {img_flat_u8x25[143:136],img_flat_u8x25[135:128],i
  ↪ mg_flat_u8x25[111:104],img_flat_u8x25[103:96]};
assign pix_blk1[5] = {img_flat_u8x25[151:144],img_flat_u8x25[143:136],i
  ↪ mg_flat_u8x25[119:112],img_flat_u8x25[111:104]};
assign pix_blk1[6] = {img_flat_u8x25[175:168],img_flat_u8x25[167:160],i
  ↪ mg_flat_u8x25[143:136],img_flat_u8x25[135:128]};
assign pix_blk1[7] = {img_flat_u8x25[183:176],img_flat_u8x25[175:168],i
  ↪ mg_flat_u8x25[151:144],img_flat_u8x25[143:136]};
assign pix_blk1[8] = {img_flat_u8x25[191:184],img_flat_u8x25[183:176],i
  ↪ mg_flat_u8x25[159:152],img_flat_u8x25[151:144]};

assign pix_blk2[0] = {24'd0, img_flat_u8x25[103:96]};
assign pix_blk2[1] = {24'd0, img_flat_u8x25[111:104]};
assign pix_blk2[2] = {24'd0, img_flat_u8x25[119:112]};
assign pix_blk2[3] = {24'd0, img_flat_u8x25[143:136]};
assign pix_blk2[4] = {24'd0, img_flat_u8x25[151:144]};
assign pix_blk2[5] = {24'd0, img_flat_u8x25[159:152]};

```

```

assign pix_blk2[6] = {24'd0, img_flat_u8x25[183:176]};
assign pix_blk2[7] = {24'd0, img_flat_u8x25[191:184]};
assign pix_blk2[8] = {24'd0, img_flat_u8x25[199:192]};

genvar gk0;
generate
    for (gk0 = 0; gk0 < 9; gk0 = gk0 + 1) begin : gen_blk0_mul
        if (USE_DSP) begin : gen_blk0_dsp
            simd_mul8x4_dsp u_mul (
                .a_u8x4(pix_blk0[gk0]),
                .b_s8x4(ker_pack[gk0]),
                .p_s16x4(prod_blk0[gk0])
            );
        end else begin : gen_blk0_booth
            simd_mul8x4 u_mul (
                .a_u8x4(pix_blk0[gk0]),
                .b_s8x4(ker_pack[gk0]),
                .p_s16x4(prod_blk0[gk0])
            );
        end
    end
endgenerate

genvar gk1;
generate
    for (gk1 = 0; gk1 < 9; gk1 = gk1 + 1) begin : gen_blk1_mul
        if (USE_DSP) begin : gen_blk1_dsp
            simd_mul8x4_dsp u_mul (
                .a_u8x4(pix_blk1[gk1]),
                .b_s8x4(ker_pack[gk1]),
                .p_s16x4(prod_blk1[gk1])
            );
        end else begin : gen_blk1_booth
            simd_mul8x4 u_mul (
                .a_u8x4(pix_blk1[gk1]),
                .b_s8x4(ker_pack[gk1]),
                .p_s16x4(prod_blk1[gk1])
            );
        end
    end
endgenerate

genvar gk2;
generate
    for (gk2 = 0; gk2 < 9; gk2 = gk2 + 1) begin : gen_blk2_mul
        if (USE_DSP) begin : gen_blk2_dsp

```

```

        simd_mul8x4_dsp u_mul (
            .a_u8x4(pix_blk2[gk2]),
            .b_s8x4(ker_pack[gk2]),
            .p_s16x4(prod_blk2[gk2])
        );
    end else begin : gen_blk2_booth
        simd_mul8x4 u_mul (
            .a_u8x4(pix_blk2[gk2]),
            .b_s8x4(ker_pack[gk2]),
            .p_s16x4(prod_blk2[gk2])
        );
    end
end
endgenerate

assign conv0 = lane20(prod_blk0[0], 0) + lane20(prod_blk0[1], 0) +
↳ lane20(prod_blk0[2], 0) +
    lane20(prod_blk0[3], 0) + lane20(prod_blk0[4], 0) +
    ↳ lane20(prod_blk0[5], 0) +
    lane20(prod_blk0[6], 0) + lane20(prod_blk0[7], 0) +
    ↳ lane20(prod_blk0[8], 0);
assign conv1 = lane20(prod_blk0[0], 1) + lane20(prod_blk0[1], 1) +
↳ lane20(prod_blk0[2], 1) +
    lane20(prod_blk0[3], 1) + lane20(prod_blk0[4], 1) +
    ↳ lane20(prod_blk0[5], 1) +
    lane20(prod_blk0[6], 1) + lane20(prod_blk0[7], 1) +
    ↳ lane20(prod_blk0[8], 1);
assign conv2 = lane20(prod_blk0[0], 2) + lane20(prod_blk0[1], 2) +
↳ lane20(prod_blk0[2], 2) +
    lane20(prod_blk0[3], 2) + lane20(prod_blk0[4], 2) +
    ↳ lane20(prod_blk0[5], 2) +
    lane20(prod_blk0[6], 2) + lane20(prod_blk0[7], 2) +
    ↳ lane20(prod_blk0[8], 2);
assign conv3 = lane20(prod_blk0[0], 3) + lane20(prod_blk0[1], 3) +
↳ lane20(prod_blk0[2], 3) +
    lane20(prod_blk0[3], 3) + lane20(prod_blk0[4], 3) +
    ↳ lane20(prod_blk0[5], 3) +
    lane20(prod_blk0[6], 3) + lane20(prod_blk0[7], 3) +
    ↳ lane20(prod_blk0[8], 3);

assign conv4 = lane20(prod_blk1[0], 0) + lane20(prod_blk1[1], 0) +
↳ lane20(prod_blk1[2], 0) +
    lane20(prod_blk1[3], 0) + lane20(prod_blk1[4], 0) +
    ↳ lane20(prod_blk1[5], 0) +
    lane20(prod_blk1[6], 0) + lane20(prod_blk1[7], 0) +
    ↳ lane20(prod_blk1[8], 0);

```

```

assign conv5 = lane20(prod_blk1[0], 1) + lane20(prod_blk1[1], 1) +
↳ lane20(prod_blk1[2], 1) +
    lane20(prod_blk1[3], 1) + lane20(prod_blk1[4], 1) +
    ↳ lane20(prod_blk1[5], 1) +
    lane20(prod_blk1[6], 1) + lane20(prod_blk1[7], 1) +
    ↳ lane20(prod_blk1[8], 1);
assign conv6 = lane20(prod_blk1[0], 2) + lane20(prod_blk1[1], 2) +
↳ lane20(prod_blk1[2], 2) +
    lane20(prod_blk1[3], 2) + lane20(prod_blk1[4], 2) +
    ↳ lane20(prod_blk1[5], 2) +
    lane20(prod_blk1[6], 2) + lane20(prod_blk1[7], 2) +
    ↳ lane20(prod_blk1[8], 2);
assign conv7 = lane20(prod_blk1[0], 3) + lane20(prod_blk1[1], 3) +
↳ lane20(prod_blk1[2], 3) +
    lane20(prod_blk1[3], 3) + lane20(prod_blk1[4], 3) +
    ↳ lane20(prod_blk1[5], 3) +
    lane20(prod_blk1[6], 3) + lane20(prod_blk1[7], 3) +
    ↳ lane20(prod_blk1[8], 3);

assign conv8 = lane20(prod_blk2[0], 0) + lane20(prod_blk2[1], 0) +
↳ lane20(prod_blk2[2], 0) +
    lane20(prod_blk2[3], 0) + lane20(prod_blk2[4], 0) +
    ↳ lane20(prod_blk2[5], 0) +
    lane20(prod_blk2[6], 0) + lane20(prod_blk2[7], 0) +
    ↳ lane20(prod_blk2[8], 0);

assign conv_blk0_s20x4 = {conv3, conv2, conv1, conv0};
assign conv_blk1_s20x4 = {conv7, conv6, conv5, conv4};
assign conv_blk2_s20x4 = {20'sd0, 20'sd0, 20'sd0, conv8};
endmodule

```

## 5.2 标准样例与手算结果

系统 testbench `tb_cnn_core` 采用如下标准输入：

- 图像矩阵为从 1 到 25 的顺序数列，按行优先排布为  $5 \times 5$ ；
- 卷积核 9 个元素均为 1。

对应的卷积结果为：

$$\begin{bmatrix} 63 & 72 & 81 \\ 108 & 117 & 126 \\ 153 & 162 & 171 \end{bmatrix}$$

ReLU 后保持不变。继续执行  $2 \times 2$  平均池化，可得到：

$$\begin{bmatrix} 90 & 99 \\ 135 & 144 \end{bmatrix}$$

在 out\_flat\_u8x4 中，该结果被打包为 32'h9087635A。

### 5.3 系统仿真结果

系统级仿真命令为：

```
powershell -ExecutionPolicy Bypass -File .\scripts\run_vivado_sim.ps1 -Mode  
↪ system
```

tb\_cnn\_core 对 5 个顶层分别拉起 start，并显式检查输出与延迟行为。  
结果如表 2 所示。

表 2: 系统级仿真结果

| 实验组                       | 输出结果         | 延迟周期 |
|---------------------------|--------------|------|
| 串行                        | 32'h9087635A | 96   |
| 串行 + BoothWallace         | 32'h9087635A | 96   |
| BoothWallace + SIMD       | 32'h9087635A | 1    |
| BoothWallace + SIMD + 流水线 | 32'h9087635A | 3    |
| DSP 替换版                   | 32'h9087635A | 3    |

XSIM 日志中可以直接看到如下 PASS 信息：

```
CASE=1 PASS cycles=96 out=9087635a  
CASE=2 PASS cycles=96 out=9087635a  
CASE=3 PASS cycles=1 out=9087635a  
CASE=4 PASS cycles=3 out=9087635a  
CASE=5 PASS cycles=3 out=9087635a  
ALL TESTS PASSED.
```

下面给出系统级 testbench 的完整 Verilog 源码。

## 系统级 testbench tb\_cnn\_core 的 Verilog 源码

对应文件: ../sim/tb\_cnn\_core.v

```
`timescale 1ns/1ps

module tb_cnn_core;
    reg clk;
    reg rst_n;
    reg start_base;
    reg start_booth;
    reg start_simd;
    reg start_pipe;
    reg start_dsp;
    reg [199:0] img_flat_u8x25;
    reg signed [71:0] ker_flat_s8x9;
    wire done_base;
    wire done_booth;
    wire done_simd;
    wire done_pipe;
    wire done_dsp;
    wire [31:0] out_base_u8x4;
    wire [31:0] out_booth_u8x4;
    wire [31:0] out_simd_u8x4;
    wire [31:0] out_pipe_u8x4;
    wire [31:0] out_dsp_u8x4;

    localparam [31:0] EXPECT = {8'd144, 8'd135, 8'd99, 8'd90};

    cnn_compare_base_top u_base (
        .clk(clk),
        .rst_n(rst_n),
        .start(start_base),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .done(done_base),
        .out_flat_u8x4(out_base_u8x4)
    );

    cnn_compare_booth_top u_booth (
        .clk(clk),
        .rst_n(rst_n),
        .start(start_booth),
        .img_flat_u8x25(img_flat_u8x25),
        .ker_flat_s8x9(ker_flat_s8x9),
        .done(done_booth),
        .out_flat_u8x4(out_booth_u8x4)
    );
endmodule
```

```

);

cnn_compare_booth_simd_top u_simd (
    .clk(clk),
    .rst_n(rst_n),
    .start(start_simd),
    .img_flat_u8x25(img_flat_u8x25),
    .ker_flat_s8x9(ker_flat_s8x9),
    .done(done_simd),
    .out_flat_u8x4(out_simd_u8x4)
);

cnn_compare_booth_simd_pipe_top u_pipe (
    .clk(clk),
    .rst_n(rst_n),
    .start(start_pipe),
    .img_flat_u8x25(img_flat_u8x25),
    .ker_flat_s8x9(ker_flat_s8x9),
    .done(done_pipe),
    .out_flat_u8x4(out_pipe_u8x4)
);

cnn_compare_booth_simd_pipe_dsp_top u_dsp (
    .clk(clk),
    .rst_n(rst_n),
    .start(start_dsp),
    .img_flat_u8x25(img_flat_u8x25),
    .ker_flat_s8x9(ker_flat_s8x9),
    .done(done_dsp),
    .out_flat_u8x4(out_dsp_u8x4)
);

always #5 clk = ~clk;

task load_img_1_to_25;
    integer i;
    begin
        img_flat_u8x25 = 200'd0;
        for (i = 0; i < 25; i = i + 1) begin
            img_flat_u8x25[i*8 +: 8] = i + 1;
        end
    end
endtask

task load_kernel_all_ones;
    integer i;

```

```

begin
    ker_flat_s8x9 = 72'd0;
    for (i = 0; i < 9; i = i + 1) begin
        ker_flat_s8x9[i*8 +: 8] = 8'sd1;
    end
end
endtask

task automatic wait_done_and_check;
    input integer case_id;
    input integer expect_cycles;
    input integer dut_sel;
    integer cycles;
    reg done_now;
    reg [31:0] out_now;
    begin
        begin : wait_loop
            cycles = 1;
            done_now = 1'b0;
            out_now = 32'd0;

            while (1'b1) begin
                case (dut_sel)
                    1: begin
                        done_now = done_base;
                        out_now = out_base_u8x4;
                    end
                    2: begin
                        done_now = done_booth;
                        out_now = out_booth_u8x4;
                    end
                    3: begin
                        done_now = done_simd;
                        out_now = out_simd_u8x4;
                    end
                    4: begin
                        done_now = done_pipe;
                        out_now = out_pipe_u8x4;
                    end
                    default: begin
                        done_now = done_dsp;
                        out_now = out_dsp_u8x4;
                    end
                endcase

                if (done_now === 1'b1) begin

```

```

        if (cycles != expect_cycles) begin
            $display("CASE=%0d FAIL cycles=%0d
                ↪ exp_cycles=%0d",
                    case_id, cycles, expect_cycles);
            $finish;
        end

        if (out_now != EXPECT) begin
            $display("CASE=%0d FAIL out=%h exp=%h", case_id,
                ↪ out_now, EXPECT);
            $finish;
        end

        $display("CASE=%0d PASS cycles=%0d out=%h", case_id,
            ↪ cycles, out_now);
        @(posedge clk);
        disable wait_loop;
    end

    @(posedge clk);
    cycles = cycles + 1;
    if (cycles > 200) begin
        $display("CASE=%0d TIMEOUT", case_id);
        $finish;
    end
end
end
end
endtask

initial begin
    clk = 1'b0;
    rst_n = 1'b0;
    start_base = 1'b0;
    start_booth = 1'b0;
    start_simd = 1'b0;
    start_pipe = 1'b0;
    start_dsp = 1'b0;
    load_img_1_to_25();
    load_kernel_all_ones();

    repeat(4) @(posedge clk);
    rst_n = 1'b1;
    @(posedge clk);

    start_base = 1'b1;

```

```

    @(posedge clk);
    start_base = 1'b0;
    wait_done_and_check(1, 96, 1);

    start_booth = 1'b1;
    @(posedge clk);
    start_booth = 1'b0;
    wait_done_and_check(2, 96, 2);

    start_simd = 1'b1;
    @(posedge clk);
    start_simd = 1'b0;
    wait_done_and_check(3, 1, 3);

    start_pipe = 1'b1;
    @(posedge clk);
    start_pipe = 1'b0;
    wait_done_and_check(4, 3, 4);

    start_dsp = 1'b1;
    @(posedge clk);
    start_dsp = 1'b0;
    wait_done_and_check(5, 3, 5);

    $display("ALL TESTS PASSED.");
    $finish;
end
endmodule

```

## 5.4 系统级与流水线波形

图 5 展示了 5 个独立顶层在同一 testbench 中的 **start/done** 时序对比，可以直观看到 96 周期、1 周期与 3 周期三类不同延迟行为。

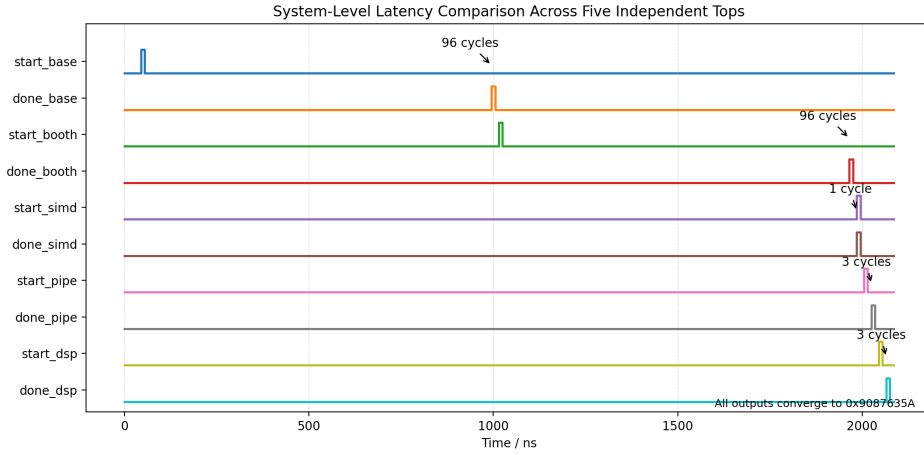


图 5: 系统级波形: 5 个独立顶层的 **start/done** 延迟对比

图 6 给出了第 4 组流水线版本的关键内部波形。可以看到 **vld\_pipe** 按 001 → 010 → 100 依次推进, 卷积结果寄存、ReLU 结果寄存和池化输出寄存依次有效, 最终在第 3 个周期输出 0x9087635A。

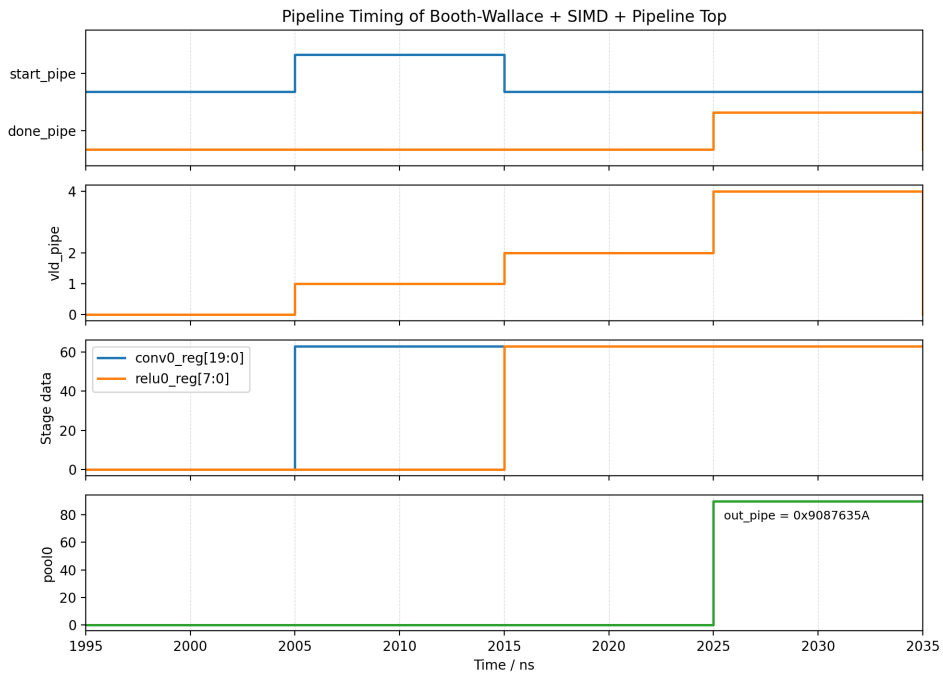


图 6: 流水线波形: 卷积寄存、ReLU 寄存与输出寄存的三级推进

## 6 里程碑 3：优化与 Vivado 性能验证

### 6.1 综合设置

综合脚本 `scripts/run_vivado_multi_synth.tcl` 会依次对以下 5 个顶层执行 `synth_design`:

- `cnn_compare_base_top`
- `cnn_compare_booth_top`
- `cnn_compare_booth_simd_top`
- `cnn_compare_booth_simd_pipe_top`
- `cnn_compare_booth_simd_pipe_dsp_top`

所有数据均来自 Vivado 2025.1 `utilization_report` 与 `timing_summary_report`。

由于本报告在 5ns 约束下比较不同 RTL 结构，因此将估算最小时钟周期写为

$$T_{\min} \approx 5.000 - \text{WNS}$$

再用

$$f_{\max} \approx \frac{1000}{T_{\min}}$$

估算最高工作频率，用于横向比较不同结构。

### 6.2 主四组资源与时序对比

主对比表仅统计前 4 组，不包含 DSP 替换版。结果如表 3 所示。

表 3: 主四组实验的资源与时序对比

| 实验组                       | LUT  | FF  | DSP | WNS/ns | 估算 $f_{\max}$ /MHz | 周期数 |
|---------------------------|------|-----|-----|--------|--------------------|-----|
| 串行                        | 550  | 304 | 0   | -5.546 | 94.82              | 96  |
| 串行 + BoothWallace         | 532  | 304 | 0   | -5.883 | 91.89              | 96  |
| BoothWallace + SIMD       | 8095 | 33  | 0   | -6.030 | 90.66              | 1   |
| BoothWallace + SIMD + 流水线 | 8097 | 287 | 0   | -1.625 | 150.94             | 3   |

结合表 3 与时序报告，可得到以下结论：

- 第 2 组在串行框架中将 LUT 从 550 降到 532，但 WNS 从 -5.546 ns 变为 -5.883 ns，说明当前关键路径仍主要受串行卷积累加与布线影响，而不是单纯由乘法器级数决定；
- 第 3 组将计算周期压缩到 1 个周期，但由于卷积、激活和池化几乎全部落在同一条组合路径上，输入端口到输出寄存器的路径最长，WNS 反而最差；
- 第 4 组加入真实三级流水后，WNS 明显改善到 -1.625 ns，估算最高频率提升到 150.94 MHz，是主四组中时钟能力最强的实现；

- 若按估算频率折算单次计算时延，则第 1/2/3/4 组约为 1012.44 ns、1044.73 ns、11.03 ns 和 19.88 ns。也就是说，第 3 组在当前结构下具有更短的单次结果延迟，而第 4 组则以更好的时钟能力和更规则的吞吐结构取胜。

| Setup                                   | Hold                               | Pulse Width                                       |
|-----------------------------------------|------------------------------------|---------------------------------------------------|
| Worst Negative Slack (WNS): -5.546 ns   | Worst Hold Slack (WHS): -0.998 ns  | Worst Pulse Width Slack (WPWS): 2.000 ns          |
| Total Negative Slack (TNS): -776.745 ns | Total Hold Slack (THS): -47.531 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 213        | Number of Failing Endpoints: 427   | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 938          | Total Number of Endpoints: 938     | Total Number of Endpoints: 305                    |

Timing constraints are not met.

图 7: 第一组实验的时序报告

| Name                     | ^1 | Slice LUTs<br>(134600) | Slice Registers<br>(269200) | Bonded IOB<br>(400) | BUFGCTRL<br>(32) |
|--------------------------|----|------------------------|-----------------------------|---------------------|------------------|
| > N cnn_compare_base_top |    | 550                    | 304                         | 308                 | 1                |

图 8: 第一组实验的综合报告

| Setup                                   | Hold                               | Pulse Width                                       |
|-----------------------------------------|------------------------------------|---------------------------------------------------|
| Worst Negative Slack (WNS): -5.883 ns   | Worst Hold Slack (WHS): -0.998 ns  | Worst Pulse Width Slack (WPWS): 2.000 ns          |
| Total Negative Slack (TNS): -827.640 ns | Total Hold Slack (THS): -47.531 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 213        | Number of Failing Endpoints: 427   | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 938          | Total Number of Endpoints: 938     | Total Number of Endpoints: 305                    |

Timing constraints are not met.

图 9: 第二组实验的时序报告

| Name                      | ^1 | Slice LUTs<br>(134600) | Slice Registers<br>(269200) | F7<br>Muxes | Bonded<br>IOB | BUFGCTRL<br>(32) |
|---------------------------|----|------------------------|-----------------------------|-------------|---------------|------------------|
| > N cnn_compare_booth_top |    | 532                    | 304                         | 2           | 308           | 1                |

图 10: 第二组实验的综合报告

| Setup                                   | Hold                               | Pulse Width                                       |
|-----------------------------------------|------------------------------------|---------------------------------------------------|
| Worst Negative Slack (WNS): -6.030 ns   | Worst Hold Slack (WHS): -1.032 ns  | Worst Pulse Width Slack (WPWS): 2.000 ns          |
| Total Negative Slack (TNS): -208.245 ns | Total Hold Slack (THS): -28.785 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 65         | Number of Failing Endpoints: 66    | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 131          | Total Number of Endpoints: 131     | Total Number of Endpoints: 34                     |

Timing constraints are not met.

图 11: 第三组实验的时序报告

| Name                           | ^1 | Slice LUTs<br>(134600) | Slice Registers<br>(269200) | Bonded<br>IOB (400) | BUFG<br>CTRL<br>(32) |
|--------------------------------|----|------------------------|-----------------------------|---------------------|----------------------|
| > N cnn_compare_booth_simd_top |    | 8095                   | 33                          | 308                 | 1                    |

图 12: 第三组实验的综合报告

| Setup                                   | Hold                                | Pulse Width                                       |
|-----------------------------------------|-------------------------------------|---------------------------------------------------|
| Worst Negative Slack (WNS): -1.625 ns   | Worst Hold Slack (WHS): -1.032 ns   | Worst Pulse Width Slack (WPWS): 2.000 ns          |
| Total Negative Slack (TNS): -142.590 ns | Total Hold Slack (THS): -165.875 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 150        | Number of Failing Endpoints: 594    | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 891          | Total Number of Endpoints: 891      | Total Number of Endpoints: 288                    |

Timing constraints are not met.

图 13: 第四组实验的时序报告

| Name                                | ^1 | Slice LUTs<br>(134600) | Slice Registers<br>(269200) | Bonded IOB<br>(400) | BUFGCTRL<br>(32) |
|-------------------------------------|----|------------------------|-----------------------------|---------------------|------------------|
| > N cnn_compare_booth_simd_pipe_top |    | 8097                   | 287                         | 308                 | 1                |

图 14: 第四组实验的综合报告

### 6.3 第 5 组：在第 4 组基础上换用 DSP

第 5 组仅作为第 4 组的 DSP 替换对照，不进入主四组主表。其比较结果如表 4 所示。

表 4: 第 4 组与第 5 组对比

| 指标                 | 第 4 组: BoothWallace + SIMD + 流水线 | 第 5 组: DSP 替换版 |
|--------------------|----------------------------------|----------------|
| LUT                | 8097                             | 1372           |
| FF                 | 287                              | 287            |
| DSP                | 0                                | 81             |
| WNS/ns             | -1.625                           | -2.054         |
| 估算 $f_{\max}$ /MHz | 150.94                           | 141.76         |
| 系统仿真周期数            | 3                                | 3              |

这一组实验的结论比较明确：

- 换用 DSP48E1 后，LUT 从 8097 大幅下降到 1372，逻辑资源减少约 83.1%；
- 但在当前 200 MHz 约束下，DSP 版的 WNS 并没有改善，反而从 -1.625 ns 变为 -2.054 ns；
- 功能与延迟周期保持不变，说明第 5 组达成了“只改实现、不改功能”的目标；
- 因此，对本工程当前 RTL 而言，DSP 替换带来的主要收益是逻辑资源迁移，而不是直接的性能提升。

| Setup                                   | Hold                                | Pulse Width                                       |
|-----------------------------------------|-------------------------------------|---------------------------------------------------|
| Worst Negative Slack (WNS): -2.054 ns   | Worst Hold Slack (WHS): -1.032 ns   | Worst Pulse Width Slack (WPWS): 2.000 ns          |
| Total Negative Slack (TNS): -308.675 ns | Total Hold Slack (THS): -153.182 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 204        | Number of Failing Endpoints: 468    | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 891          | Total Number of Endpoints: 891      | Total Number of Endpoints: 288                    |

Timing constraints are not met.

图 15: 第五组实验的时序报告

| Name                                    | ^1 | Slice LUTs<br>(134600) | Slice Registers<br>(269200) | DSPs<br>(740) | Bonded<br>IOB | BUFGCTRL<br>(32) |
|-----------------------------------------|----|------------------------|-----------------------------|---------------|---------------|------------------|
| > N cnn_compare_booth_simd_pipe_dsp_top |    | 1372                   | 287                         | 81            | 308           | 1                |

图 16: 第五组实验的综合报告

## 7 实验收获

- 完整实现并验证了 8 位定点卷积、ReLU 激活和平均池化三段链路，对 CNN 基础算子在 FPGA 上的组织方式有了更具体的理解。
- 通过把 5 组实验拆成独立顶层，进一步体会到“便于综合分析的结构”与“便于功能复用的结构”并不总是同一件事。
- BoothWallace 乘法器、SIMD 路径和 3 级流水线都已经在工程中落地，并通过统一 testbench 验证了功能一致性。
- 从 Vivado 报告中可以看出，优化并不总会直接转化为更高主频；关键路径位置、寄存器边界和布线代价同样会改变最终结果。
- DSP 替换版说明了另一类典型工程取舍：它显著降低了 LUT 占用，但在当前结构下并未直接带来性能收益。

## 8 附：源码目录

- project1/src/baseline/: 串行基线实现，包括 mul8.v、add8.v、relu\_s20.v、div4.v、cnn\_chain\_core\_base.v 等；
- project1/src/optimized/: BoothWallace、SIMD、流水线和 DSP 版本实现；
- project1/sim/: 模块级与系统级 testbench；
- project1/scripts/run\_vivado\_sim.ps1: Vivado/XSIM 仿真入口；
- project1/scripts/run\_vivado\_multi\_synth.tcl: 5 个顶层的综合入口；
- project1/scripts/export\_wave\_figures.py: 从 VCD 导出报告波形图；
- project1/constraints/project1.xdc: 时钟与 I/O 约束；
- project1/report/figures/: 报告中引用的代表性波形图片。